

Implementation spec: GRU + XGBoost market-neutral strategy

Global conventions

- **Inputs:** `chars_final_with_names.parquet` (unique on `permno`, `eom`), `8k_20150101_20260831_identified.parquet`, `factor_char_list.csv` (column variable = the 147 features). Also FRED TB3MS and the S&P 500 (monthly returns from month-end closes).
- **Alignment:** a row at `eom = t` uses data from month `t` only. The target is `ret_exc_lead1m` (excess return of month `t+1`). Never shift it again, and never use it as a feature. `ret/ret_exc` belong to month `t`.
- **Holdings date** = first day of month `t+1`. The test period covers return months 01/2021–08/2026.
- **Universe:** it is defined from month-`t` data only. Training rows need a non-missing target; prediction rows do not.
- **Schedule (by target month):**

Test year	Train	Validation
Y	2015-02 to (Y-3)-12	(Y-2)-01 to (Y-1)-12

2026 covers January–August only.

- **Capital convention:** \$1 of capital; weights w_i are signed fractions of capital.
 - excess return = $\sum w_i \cdot \text{ret_exc_lead1m}_i$
 - total return = $\text{TB3MS}_{\{t+1\}}/1200 + \text{excess}$
 - active return = $\text{excess} - 0.04/12$
- **Reproducibility:** fixed seed 42; all intermediate results cached as parquet in `./cache`, outputs in `./output`, and every setting in one config dict at the top of `MAIN.py`.
- **Hardware:** the code runs on an ASUS Ascent GX10 (ARM64 with CUDA), so use aarch64-compatible builds of torch, xgboost and vLLM.

First run

1. **Load.** Read the three files and add `target_month = eom + 1 month`. From the 8-K file, keep `document_id`, `permno`, `filing_date`, `items`, `text`, `text_sha256`. Deduplicate filings on (`permno`, `text_sha256`).

2. **Universe (per eom, month-t data only).**

- Keep stocks with `abs(prc) ≥ 5` and `market_equity` above that month's 20th percentile.
- **Short-eligible subset:** additionally, `market_equity` at or above the 40th percentile, `dolvol_126d` at or above the universe's 30th percentile, and `zero_trades_126d` at or below the universe median.
- The thresholds are defaults, tunable under step 10.

3. **Preprocess.**

- Each month, rank-transform all 147 features to `[-1, 1]` (the template's method), then fill missing values with 0 (the median).
- Add missing-value flags for features that are more than 20% missing in the training data.
- `sector` = first 2 digits of `gics`.

4. **GRU encoder (unsupervised).**

- **Input:** for each (`permno`, `t`), the 11 month-to-month changes of the ranked features over `t-11..t`, shape (11, 147). Require at least 6 valid months; otherwise the embedding is NaN.
- **Model:** a 1-layer GRU encoder (hidden size 64), then a linear layer to `d` dimensions. A GRU decoder reconstructs the input sequence, trained with MSE, early-stopped on 2019–2020 reconstruction loss.

- **Optimizer:** AdamW, with weight decay chosen from $\{0, 1e-4, 1e-3\}$ by 2019–2020 reconstruction loss.
- **Gradient clipping:** clip the gradient norm at 1.0 on every step (`torch.nn.utils.clip_grad_norm_`).
- **Train** only on sequences with `eom ≤ 2018-11`, then freeze.
- **Output:** d embedding columns `gru_1..gru_d`, with $d \in \{8, 16, 32\}$ chosen under step 10.

5. XGBoost ranker.

- `XGBRanker(objective="rank:pairwise")` with `qid = eom`, rows sorted by `eom`.
- **Label:** the within-month decile (0–9) of `ret_exc_lead1m`.
- **Features:** ranked features, missing flags, `gru_*`, and the 8-K columns from steps 6 and 7.
- **Fixed settings:** `learning_rate=0.03`, `subsample=0.7`, `colsample_bytree=0.5`, `min_child_weight=200`, `reg_lambda=10`, `tree_method="hist"`.
- **Grid:** `max_depth` $\in \{3, 4, 5\}$, up to 2,000 trees. Evaluate the validation mean monthly rank IC every 50 trees and pick the best (depth, trees).
- **Refit** on train + validation with those settings and predict the test year.
- **Return forecast (for OOS R^2):** on the validation years, using the scores from the train-only model fitted during tuning, fit a linear regression of `ret_exc_lead1m` on the within-month rank of the score (scaled to $[-1, 1]$). Apply it to each test year's score ranks to get a return forecast `ret_hat`.
- **Output:** `predictions.parquet` with columns (`permno`, `eom`, `score`, `ret_hat`).

6. 8-K event flags.

- Assign each filing to the `eom` of its `filing_date` month, and only to that month.
- **Per (`permno`, `eom`):** `has_filing`, `n_filings`, and indicators for items 1.01, 1.02, 1.03, 2.01, 2.02, 2.03, 2.05, 2.06, 3.01, 4.01, 4.02, 5.02, 7.01 and 8.01.
- **History counts:** trailing 12-month counts, covering months $\leq t$, of 4.02, 5.02, 2.06, 1.02 and 2.05.
- Stock-months without a filing get `has_filing=0`, and their other text columns are NaN.

7. FinBERT tone.

- **Model:** ProsusAI/finbert, whose pretraining predates 2021.
- **Anonymize:** before encoding, remove the firm's `company_name` and `ticker` from the text (case-insensitive).
- **Chunking:** 512-token chunks, at most 8 per filing. Per chunk, `tone = P(positive) – P(negative)`.
- **Per filing:** the mean and the minimum over chunks.
- **Per stock-month:** `tone_mean`, `tone_min`, and `tone_surprise = tone_mean` minus the firm's trailing 12-month average, using months before `t` only.
- Cache results by `document_id`.

8. Kalman betas.

- **Observations per stock-month:** `beta_dimson_21d`, `betadown_252d`, `betabab_1260d`, `beta_60m`.
- **State:** the true beta, with $\beta_t - m_t = \phi(\beta_{t-1} - m_{t-1}) + \text{noise}(q)$. The prior m_t is the sector median of `beta_60m` at `t`.
- Sequential scalar updates, one per available observation, each with its own noise variance R_j .
- Estimate ϕ , q and the R_j by maximum likelihood on `eom ≤ 2018-12`, pooled across stocks, then freeze.
- **Output:** `beta_kf`, `beta_var`. If the filter has no data, fall back to `beta_60m`.

9. CVXPY optimizer (monthly).

- **Candidates:** the top 250 by score from the universe (longs) and the bottom 250 from the short-eligible subset (shorts). Standardize the scores within the month.
- **Variables:** $w_L \geq 0$ over long candidates and $w_S \geq 0$ over short candidates. Let $w = w_L - w_S$.
- **Objective:** maximize $s \cdot w - \lambda_{tc} \cdot \|w - w_{\text{prev}}\|_1 - \lambda_{\beta} \cdot \sum \text{beta_var}_i \cdot w_i^2$. In `w_prev`, names that dropped out of the candidate set count as selling to 0.
- **Constraints:**
 - $\sum w_L + \sum w_S = 2$ (200% gross)
 - $-0.20 \leq \sum w_L - \sum w_S \leq 0.20$ (net exposure)

- $|\text{beta_kf} \cdot w| \leq 0.02$ (beta)
- each sector's net exposure within ± 0.10
- every weight ≤ 0.015
- These guarantee at least 120 names (each side holds at least 0.9 of capital at ≤ 0.015 per name) and at most 500.
- **Solver:** Clarabel. If a month is infeasible, relax the beta tolerance in steps of 0.01 and log it.
- **Missing returns:** if a held stock's `ret_exc_lead1m` is missing, first check whether the readme defines delisting returns. If it doesn't, use 0 and report how often this happened.

10. Settings rule.

- Nothing is chosen using 2021–2026 results.
- **XGBoost:** tune per window (step 5).
- **Fixed once on 2019–2020, then frozen:** the GRU size d , λ_{tc} , λ_{β} and the universe thresholds. Choose them by the validation-period information ratio of the full pipeline, trained on data through 2018.
- **Beta check before any test run:** regress the 2019–2020 validation book's monthly returns on the S&P 500. If the realized beta is clearly away from zero, fix the beta estimates before touching 2021–2026. Log the result.
- Write every grid and choice to `output/settings_log.json`.

11. Leakage tests.

- Assert every filing used has `filing_date` $\leq$ `eom`.
- Assert all training target months come before each test year.
- Assert no feature is identical to the target.
- Assert `(permno, eom)` is unique.
- **Shuffled-label run:** permute targets within each month; the test IC and IR must be close to 0.
- Save the results to `output/leakage_tests.json`.

12. Ablations (same schedule for all; report validation and test rank IC, ICIR, IR and beta):

1. The template's Ridge on the 147 features
2. XGBoost on the 147 features
3. + GRU
4. + event flags
5. + FinBERT

For the 8-K columns, also report the IC on stock-months with `has_filing=1`.

13. Performance pack (test period 01/2021–08/2026).

- **Statistics:** every item in the brief's list:
 - OOS R^2 of `ret_hat` over the test period, using the brief's formula with a zero benchmark: $1 - \frac{\sum (r - \hat{r})^2}{\sum r^2}$;
 - returns: arithmetic and geometric (CAGR), cumulative, calendar-year (strategy, benchmark, S&P 500), best and worst month, hit rate;
 - long and short legs separately;
 - IR, Sharpe;
 - alpha and beta versus the S&P 500, with Newey-West errors (3 lags), from excess returns on market excess returns;
 - correlation with the S&P 500;
 - holdings count per leg;
 - gross and net exposure (average and range);
 - largest weight and top-10 share;
 - turnover (average and range);
 - short-book market cap, dollar volume and small-cap share.

- **Charts:** cumulative return (strategy, benchmark, S&P 500); drawdown with the S&P 500 overlaid; rolling 12-month active return and IR; rolling 12-month beta; histogram of returns with the hurdle marked; top-10 positive and negative contributors by ticker and name.
- State that the FRED S&P 500 is a price index without dividends, and that returns are gross of trading costs.

14. Submission files.

- `output/holdings.csv`: Date, PERMNO, TICKER, COMPANY NAME, WEIGHT. WEIGHT is a percentage of NAV: positive for longs, negative for shorts.
 - Fill missing labels from the 8-K file at the nearest date, and log any stock that stays unlabeled.
- `output/returns.csv`: Date, total, excess, active, long_leg, short_leg, benchmark, sp500.
- `MAIN.py`: a single file that runs steps 1–14 end to end and ships no data.

After the first run (optional)

A. Local LLM agent (gpt-oss-20b served by vLLM on the GX10).

- Measure throughput on 1,000 filings first.
- **Triage** runs on every filing and returns JSON: material yes/no, event type, severity 0–3.
- **For material filings only:**
 - an item-specific JSON checklist, with each fact paired with a verbatim supporting quote;
 - a history tool that returns the same permno's filings with `filing_date <` the current one.
- **Verifier (code, not an LLM):** after normalizing whitespace, line breaks and quotation marks, each quote must appear in the filing text. Facts that fail are dropped.
- Anonymize the text as in step 7. Batch requests, cache by `document_id`, and aggregate the facts into stock-month columns for XGBoost.

B. Cross-attention reader (needs A).

- **Tokens:** each fact becomes an embedding of its fact type plus its values. The query is a linear projection of the stock's 147 ranked features.
- **Model:** one multi-head cross-attention layer, then a 16-dimensional bottleneck, then two 3-class heads:
 - month-t return tercile relative to the sector (weight 1.0);
 - month t+1 tercile (weight 0.3).
- **Training:** cross-entropy with label smoothing 0.1, on the same schedule, early-stopped on validation IC.
- **Outputs:** $P(\text{top}) - P(\text{bottom})$ for each head, plus the bottleneck, become XGBoost columns. Save the attention weights for the explainability slide.

C. Look-ahead proof for A.

- Run the same checklist with Llama 3.1 8B on 2,000 filings and report the agreement rate.
- **Fact-flip test:** on 200 filings, edit one key fact (for example, "raised" becomes "lowered") and check that the extraction changes accordingly.

D. Extra ablations: the first run versus + A versus + A + B, using the metrics from step 12.